SWINBURNE VIETNAM

HO CHI MINH CAMPUS

COS30018 - COS30018-Intelligent Systems

Oral Defense week 4-5

INSTRUCTOR: MR. KY TRUNG PHAM

STUDENT NAME: LÊ TIẾN DŨNG-SWS00683

HO CHI MINH CITY –2026

Contents

Oral Defense week 4 3

1. Explain the differences you observed between the initial code (v0.1) and the GitHub project (P1). Which one provided better initial results, and how did you define "better"? 3

2. Why is it highly recommended to use a virtual environment for this project, and what are the key libraries included in your requirements.txt? 3

3. In v0.1, the training and testing dates are manually hardcoded. What are the risks of this approach when trying to evaluate a model's performance over different time periods? 4

Oral Defense week 5 5

1. Your new data loading function must handle "multiple features." Beyond just the 'Close' price, how do features like 'Volume' or 'High/Low' provide extra context to a Deep Learning model? 5

2.How does your system handle NaN (Not a Number) values in the historical stock data? Why is it better to handle these during preprocessing rather than letting the model process them? 5

3.Explain the logic of scaling your data (e.g., using MinMaxScaler). Why must we store the scaler object to "inverse transform" the results later? 7

Oral Defense week 4

1. Explain the differences you observed between the initial code (v0.1) and the GitHub project (P1). Which one provided better initial results, and how did you define "better"?

The main difference between v0.1 and P1 lies in implementation robustness and prediction behavior. The v0.1 version is a simpler baseline LSTM model which uses fixed training and testing dates. The model produces smoother predictions which exhibit time delays when compared to actual price movements. P1 required dependency fixes to run properly but its predictions matched price trends better because it tracked short-term price movements more accurately.

P1 provided better initial results. In this context, "better" was defined as producing predictions with lower expected errors, reduced temporal lag, and closer visual alignment with actual stock price movements such as:

- MAE (Mean Absolute Error)
- MSE / RMSE
- MAPE

A model is considered better if it yields lower error metrics such as MAE, RMSE, or MAPE on the test dataset.

2. Why is it highly recommended to use a virtual environment for this project, and what are the key libraries included in your requirements.txt?

The project needs a virtual environment because its code requires specific machine learning and data processing library versions which will clash with the system's existing globally installed packages. The virtual environment provides three main benefits which include separating dependencies and enabling users to replicate their work across various machines and execution

times. TensorFlow-based projects require this aspect because version mismatches lead to both runtime errors and compatibility issues.

The project needs to use NumPy and Pandas as its main numerical computation and data manipulation libraries while it requires Matplotlib for graphics and TensorFlow to build and train an LSTM model and scikit-learn for data preprocessing and scaling and pandas-datareader and yfinance to obtain financial time-series data and Jupyter/ipykernel for interactive development and testing.

```
numpy
matplotlib
pandas
tensorflow
yfinance
mplfinance
scikit-learn
notebook
ipykernel
```

3. In v0.1, the training and testing dates are manually hardcoded. What are the risks of this approach when trying to evaluate a model's performance over different time periods?

Manually hardcoding training and testing dates introduces several risks when evaluating model performance. First, it reduces flexibility, as the model cannot be easily tested on different time ranges without modifying the source code. Second, this approach increases the risk of biased evaluation, since the selected dates may unintentionally favor certain market conditions and fail to represent broader trends or volatility. Finally, hardcoded dates limit reproducibility and scalability, making it difficult to perform systematic experiments such as rolling-window evaluation or cross-period validation, which are essential for reliable assessment of time-series prediction models.

Oral Defense week 5

1. Your new data loading function must handle "multiple features." Beyond just the 'Close' price, how do features like 'Volume' or 'High/Low' provide extra context to a Deep Learning model?

In my loader I use five features: Open, High, Low, Close, Volume (so n_features = 5 and the model input becomes (samples, 60, 5)).

Using only Close loses a lot of market context. Adding features helps because:

- High/Low capture the intraday trading range (volatility). Two days can have the same Close, but very different High/Low, which tells the model if price movement was stable or noisy.
- Open helps represent gaps (difference between yesterday's Close and today's Open) and early session direction.
- Volume reflects strength/participation behind price moves. A price rise on high volume can be more meaningful than the same rise on low volume.

So multi-feature inputs let the LSTM learn patterns about trend + volatility + market activity, not just the ending price.

2.How does your system handle NaN (Not a Number) values in the historical stock data? Why is it better to handle these during preprocessing rather than letting the model process them?

In my function load_and_process_multi_feature_dataset(...), I explicitly handle missing/invalid values before building sequences and training:

```python
def load_and_process_multi_feature_dataset(
    company: str,
    dataset_start: str,
    dataset_end: str,
    feature_columns,
    target_column: str = "Close",
    prediction_days=60,
    # (c) Split methods:
    split_method="date",   # "ratio" | "date" | "random"
    train_ratio=0.8,
    split_date=None,
    random_seed=42,
    # (b) NaN handling:
    nan_strategy="ffill_bfill",  # "ffill_bfill" | "drop" | "interpolate" | "ffill" | "bfill"
    # (e) Scaling options:
    scale_features=True,
    feature_range=(0, 1),
    # (d) Local caching:
    cache_dir="data_cache",
    use_cache=True,
):
    """
```

1. I first convert invalid numbers to missing values:

- df.replace([np.inf, -np.inf], np.nan, inplace=True)

```python
    # Replace +/-inf with NaN so we can handle all missing/invalid values consistently
    df.replace([np.inf, -np.inf], np.nan, inplace=True)
```

2. Then I apply a configurable NaN strategy:

- "drop" → remove rows with NaNs
- "ffill" / "bfill"
- "ffill_bfill" (my default) → forward-fill then back-fill
- "interpolate" (time interpolation)

```python
    # 2) Handle NaNs (Requirement b)
    # ----------------------------
    if nan_strategy == "drop":
        df = df.dropna()
    elif nan_strategy == "ffill":
        df = df.ffill()
    elif nan_strategy == "bfill":
        df = df.bfill()
    elif nan_strategy == "ffill_bfill":
        # Common time-series baseline:
        # - forward fill uses last known value
        # - back fill fixes NaNs at the start of the series
        df = df.ffill().bfill()
    elif nan_strategy == "interpolate":
        # Time interpolation can be helpful for small gaps; we still ffill/bfill as a safety net
        df = df.interpolate(method="time").ffill().bfill()
    else:
        raise ValueError("nan_strategy must be one of: drop, ffill, bfill, ffill_bfill, interpolate")
    print("NaN before:", df_raw.isna().sum().sum())
    print("NaN after :", df_clean.isna().sum().sum())
    # Final safety: remove any remaining NaN rows
    df = df.dropna()
    df_clean = df

    if len(df_clean) <= prediction_days + 5:
        raise ValueError("Not enough data after cleaning to build sequences. Try a larger date range.")
```

Finally, I do a safety clean: df = df.dropna() before creating sequences.

Why preprocessing is better than letting the model handle NaNs:

- Neural networks and scalers cannot train properly with NaNs—NaNs often propagate and cause the loss to become NaN, breaking training.
- Handling NaNs early ensures the LSTM receives stable, numeric sequences of fixed shape.
- It also prevents hidden errors during scaling/sequence creation and makes results reproducible.

3.Explain the logic of scaling your data (e.g., using MinMaxScaler). Why must we store the scaler object to "inverse transform" the results later?

I scale the dataset using MinMaxScaler(feature_range=(0,1)) for both:

- feature columns (feature_scaler)
- target column (target_scaler)

Logic of scaling:

- Features have very different magnitudes (e.g., Volume can be millions while prices are ~80–120). Without scaling, large-magnitude features dominate the gradients and make training unstable.
- MinMax scaling compresses each feature into a consistent range [0,1], which usually helps LSTM convergence and makes learning smoother.

Why store scalers (and fit on train only):

- I fit scalers on the training portion only (to avoid test leakage), then transform the full dataset using those scalers.
- The model predicts in scaled space, so to report real prices I must inverse-transform:
    - predicted_prices = target_scaler.inverse_transform(predicted_scaled)
    - next_price = target_scaler.inverse_transform(next_scaled)
- That’s why I store scalers in a dictionary:
    - scalers["feature_scaler"], scalers["target_scaler"], plus the column lists for reproducibility.

Without storing scalers, I couldn’t convert predictions back into real dollar price units, and the output would remain meaningless 0–1 value.

```python
    x_scaler = MinMaxScaler(feature_range=feature_range) if scale_features else None
    y_scaler = MinMaxScaler(feature_range=feature_range) if scale_features else None

    # For ratio/date split, "train rows" are all rows before split_idx.
    # For random split, we will compute train rows after selecting train sequences.
    if split_method in {"ratio", "date"}:
        train_df = df_clean.iloc[:split_idx]

        if scale_features:
            x_scaler.fit(train_df[feature_columns].values)
            y_scaler.fit(train_df[[target_column]].values)

            X_scaled = x_scaler.transform(df_clean[feature_columns].values)
            y_scaled = y_scaler.transform(df_clean[[target_column]].values).reshape(-1)
        else:
            X_scaled = df_clean[feature_columns].values
            y_scaled = df_clean[target_column].values.astype(float)
```